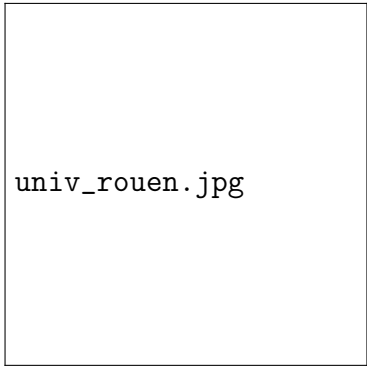




univ_rouen.jpg

Master 1 GIL - XML
Projet : Service REST & Compléments

RAPPORT MINI PROJET

Version : 1.0

Date : 15/04/2026

Rédigé par : Umm-Habibah Ouattara et Florian Pépin

1 Introduction

Ce projet porte sur la réalisation du service REST SEPA26. L'objectif est de construire une architecture basé sur la norme ISO 20022 fourni en cours pour gérer des transactions. Le déploiement est automatisé sur une VM via Jenkins. Il est réalisé par Umm-Habibah Ouattara et Florian Pépin.

2 Architecture du Projet

2.1 Stack Technique

Le service SEPA26 repose sur une architecture Java 21. La stack technique mis en place est le suivant :

- **Framework** : Spring Boot. Spring Web est utilisé pour l'exposition des API REST et Spring Data JPA pour la persistance des données.
- **Gestion des dépendances et Build** : Maven.
- **Stockage des données** : MariaDB et H2 (base de données embarquée pour les tests).
- **Traitement XML** : Les échanges se font au format XML en respectant la norme ISO 20022. Le projet intègre JAXB et Jackson-XML pour la sérialisation/désérialisation, ainsi que Saxon-HE pour des manipulations avancées (XSLT/XPath).
- **Interface (Front-end)** : Le projet utilise Thymeleaf pour générer les pages web. Cela permet de transformer et d'afficher les données Java/XML en pages HTML.

2.2 Points d'accès (Endpoints)

L'API est organisée autour de la ressource "Transaction". Le tableau suivant récapitule les principaux points d'entrée du service sous le préfixe `/sepa26` :

Méthode	Endpoint	Description / Format de sortie
POST	<code>/insert</code>	Création d'un virement (Flux XML ISO 20022)
DELETE	<code>/delete/{id}</code>	Suppression d'une transaction (Retour XML)
GET	<code>/resume/xml</code>	Liste de toutes les transactions en XML
GET	<code>/resume/html</code>	Liste de toutes les transactions en HTML
GET	<code>/xml/{id}</code>	Détail complet d'une transaction (XML)
GET	<code>/html/{id}</code>	Détail complet d'une transaction (HTML)
GET	<code>/search</code>	Recherche avancée de transactions (XML)

TABLE 1 – Récapitulatif des services REST

En complément, l'application propose des pages web classiques (`/`, `/help`, `/transfert`) pour la navigation utilisateur.

3 Accès aux Services (VM)

L'ensemble des services est déployé sur une machine virtuelle dédiée. Voici les informations nécessaires pour y accéder :

3.1 Informations de la VM

- **Hostname** : `vm-pepinflo-xml.univ-rouen.fr`
- **Adresse IP** : `10.130.162.188`

3.2 Identifiants des utilisateurs

Les comptes suivants ont été configurés pour l'accès et les tests :

Usage	Utilisateur	Mot de passe
Admin	<code>spring</code>	<code>abcd1230</code>
Projet REST	<code>user-git</code>	<code>user-git</code>
Tests sur la VM	<code>test</code>	<code>test</code>

TABLE 2 – Identifiants de la VM

3.3 Accès Jenkins

Le serveur Jenkins est accessible via :

- **URL** : `http://10.130.162.188:8080`
- **Login** : `jobadmin`
- **Mot de passe** : `JobAdmin26`

3.4 Service REST SEPA26

L'application est exposée sur le port **8100** et prête à recevoir des requêtes :

- **Interface Web** : `http://10.130.162.188:8100/`

4 Répartition du Travail

Le développement du projet a été équitablement réparti entre les deux membres du binôme. Le tableau ci-dessous détaille la distribution des modules selon les sections du cahier des charges :

Développeur	Modules et Tâches réalisées
Florian Pépin	— Développement des contrôleurs principaux (Accueil I.1, Aide I.2, Résumés I.3, Détails I.4 et Insertion I.5). — Bonus : Gestion optimisée des messages d'erreurs (II.1). — Infrastructure : Mise en place de la persistance (Base de données II.3).
Umm-Habibah Ouattara	— Mise en œuvre des contraintes du service et validation XSD (I.6). — Développement de l'API de recherche (II.2). — Qualité : Tests d'intégration et création de la collection Postman (II.3). — Outil : Développement de l'application de transfert de flux (III.1).

TABLE 3 – Répartition détaillée des tâches

5 Configuration de Jenkins

Pour le déploiement de notre service REST, nous avons utilisés Jenkins. Jenkins est un outils pour gérer la CI/CD d'un projet. Voici comment nous avons configurés l'outil :

- Tout d'abord nous avons configurer, la VM de sorte qu'elle contienne notre projet, et que ce dernier soit suivi par git. Par conséquent, à chaque push, le projet sur la VM est automatiquement mis à jour.
- Nous avons ensuite créer plusieurs jobs :
 - Création de l'exécutable du projet (.war ou .jar)
 - Lancement du projet via docker
 - À chaque mise à jour du code, lancement des jobs de manière automatique

5.1 Améliorations de la configuration Jenkins

Nous avons effectués quelques améliorations des jobs précédents :

- Pour le job d'exécution, avant la création du .war ou du .jar, nous avons défini une règle qui nous permet de lancer les tests unitaires et d'intégrations. Il est très important de noter ici, que c'est bien maven qui fait ces tests.

- Ensuite, pour ce qui est de Docker, nous avons fait en sorte d’y intégrer directement une base de données MariaDB ainsi que 2 transactions SEPA

6 Éléments Optionnels

Nous avons ajouté quelques fonctionnalités pour rendre le service plus simple à utiliser depuis un navigateur :

- **Formulaire d’insertion** : Il est possible d’envoyer un fichier XML directement depuis la page `/transfert` sans avoir à passer par une commande `curl`.
- **Bouton supprimer** : Sur la page des résumés (`/resume/html`), nous avons ajouté un bouton pour supprimer une transaction en un clic. C’est plus pratique que de forger une requête `DELETE` à la main.
- **Collection Postman** : Nous avons fourni le fichier `sepa26_pepin_ouattara.json` qui contient déjà toutes les requêtes prêtes à l’emploi pour tester le serveur (VM).

7 Conclusion

Le projet remplit l’ensemble des objectifs fixés. L’utilisation de Spring Boot couplée aux technologies XML (XSD, JAXB) a permis de construire un service robuste, déployé de manière sérieuse sur une VM avec une chaîne CI/CD fonctionnelle.